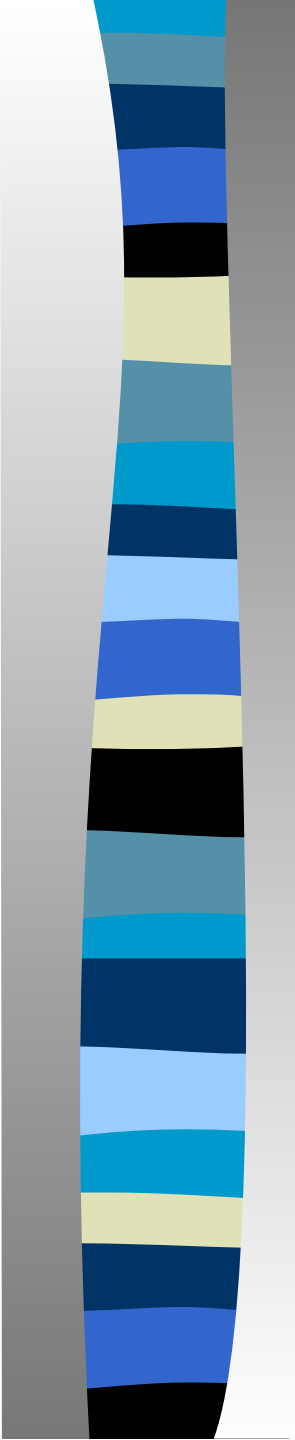
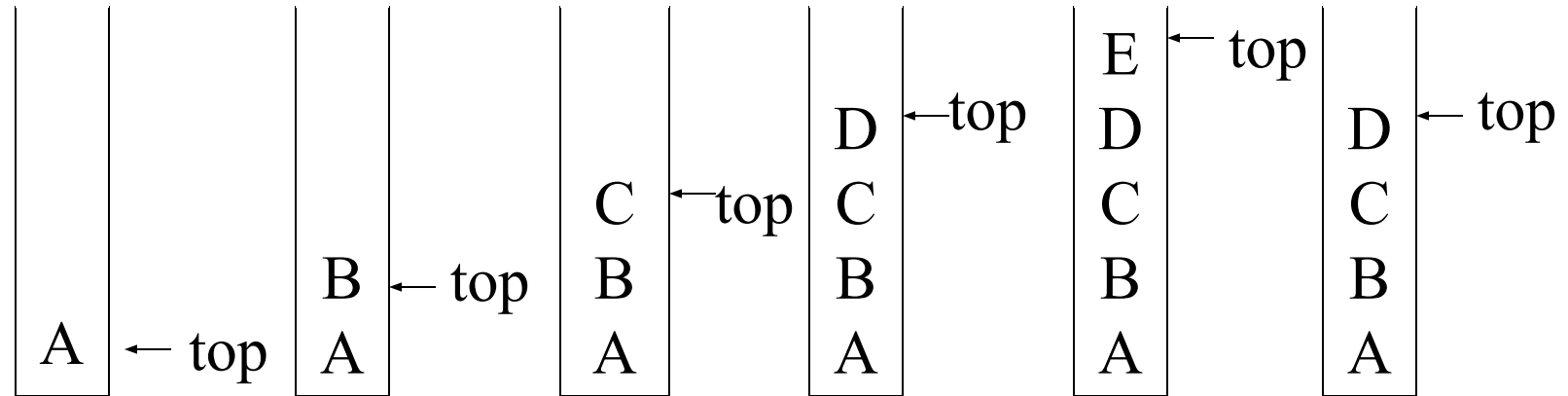


UNIT 3

STACKS

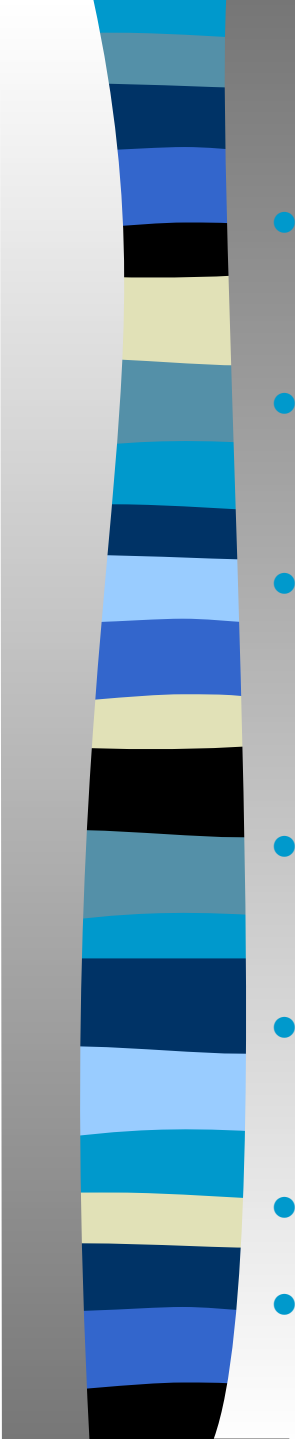
- 
- In this chapter we are going to see two data types that are frequently found in computer science.
 - These data types are, the [stack](#) and the [queue](#), which are the special cases of the more general data type, [ordered list](#).
 - A [stack](#) is an ordered list in which insertions and deletions are made at one end called the *top*.
 - Given a stack $S = \{a_0, \dots, a_{n-1}\}$, we say that a_0 is the bottom element, a_{n-1} is the top element and a_i is on top of element a_{i-1} , $0 < i < n$.
 - The restrictions on the stack imply that if we add the elements A, B, C, D, E to the stack, in that order, then E is the first element we delete from the stack.
 - Figure 3.1 illustrates this sequence of operations.
 - Since the last element inserted into a stack is the first element removed, a stack is also known as a [Last-In-First-Out \(LIFO\)](#) list.

stack: a Last-In-First-Out (LIFO) list



***Figure 3.1: Inserting and deleting elements in a stack (p.102)**

Applications of Stacks

- 
- Wearing/Removing Bangles, Pile of Dinner Plates, Stacked chairs.
 - Message logs and all messages you get are arranged in a stack.
 - Call logs, E-mails, Google photos' any gallery, YouTube downloads, Notifications (latest appears first).
 - Scratch card's earned after Google pay transaction.
 - Forward-backward surfing in the browser.
 - History of visited websites.
 - Loading bullets into the magazine of a gun. The last one to go in is fired first.
 - Media playlist. To play previous and next song
 - Undo/Redo Functionality in text editors & video editing software.
 - Java Virtual Machine.
 - Recursion.
 - Used in IDEs to check for proper parentheses matching
 - Expression Evaluation
 - Memory management
 - Back tracking
 - Parenthesis Checking
 - String Reversal
 - Function Call

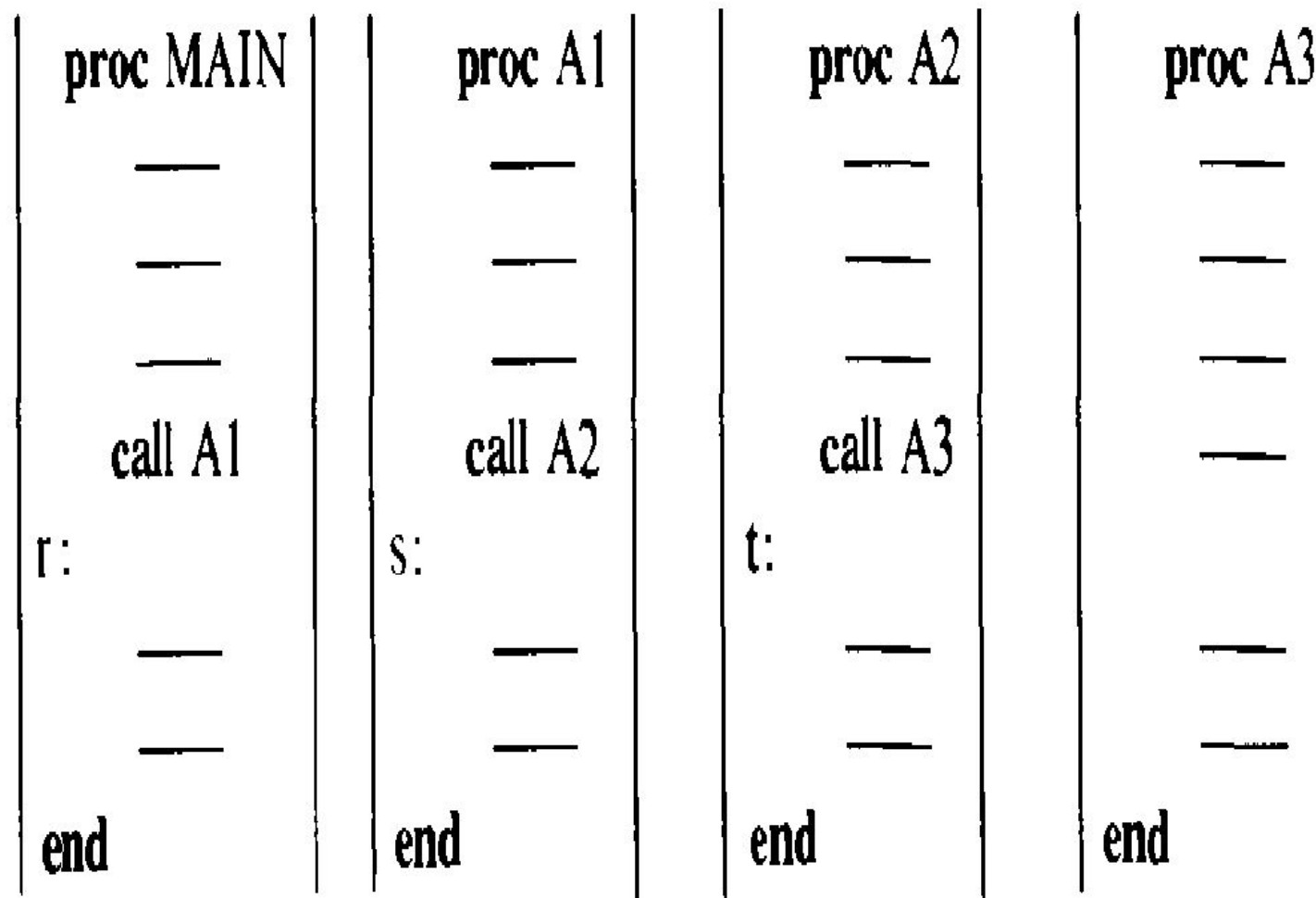
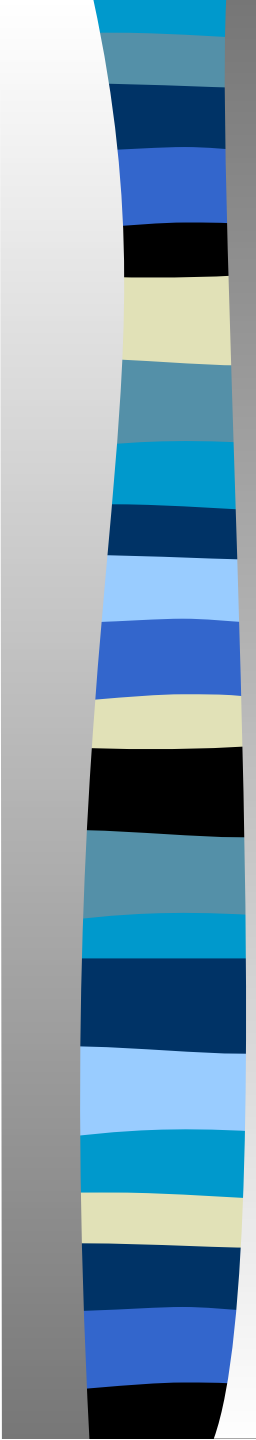
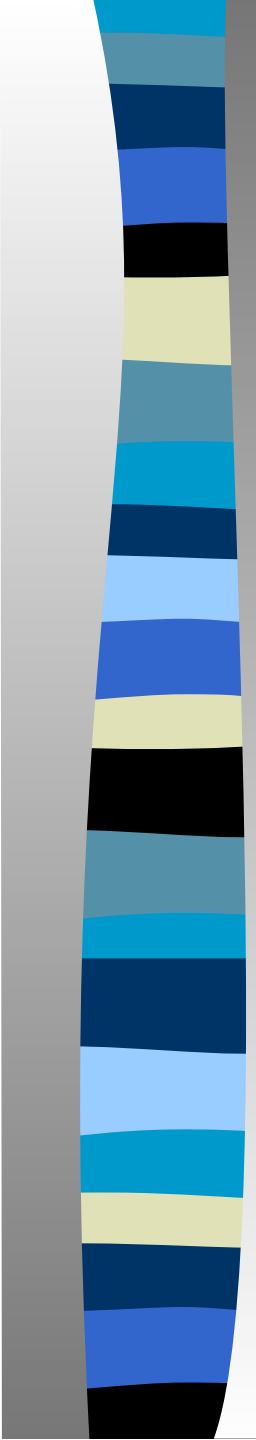
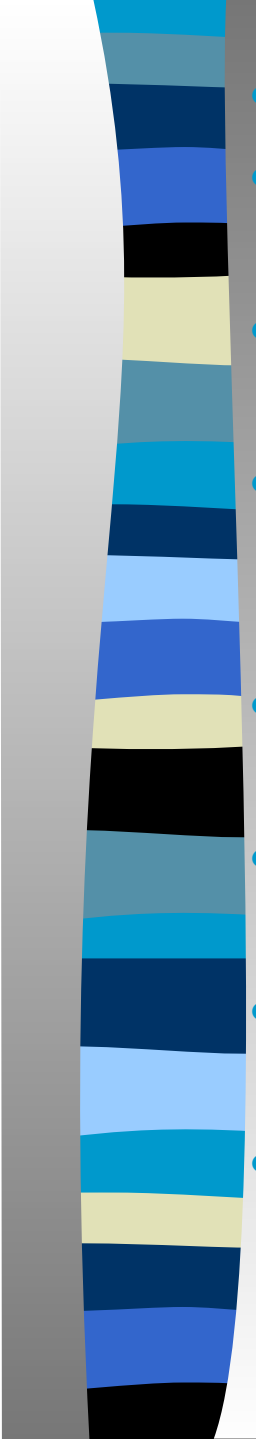
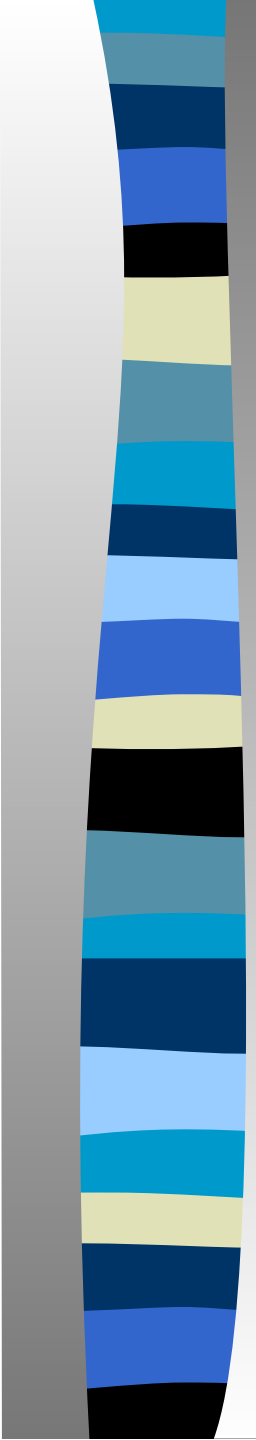


Figure 3.2. Sequence of subroutine calls

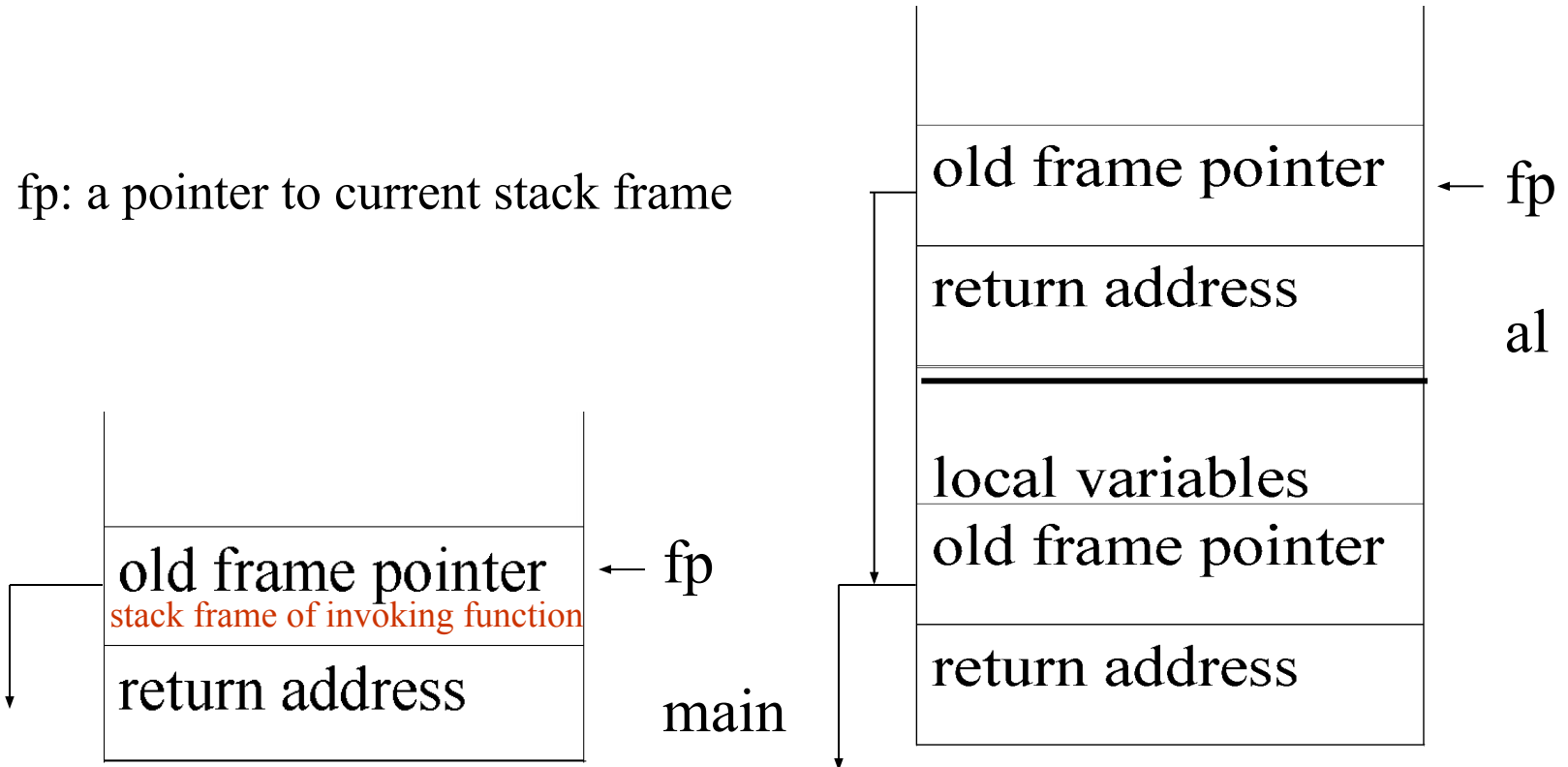
- 
- One natural example of stacks which arises in computer programming is the **processing of subroutine calls and their returns**.
 - Suppose we have a **main procedure** and **three subroutines** as below:
 - The **MAIN** program calls **subroutine A1**. On **completion** of **A1** execution of **MAIN** will **resume** at location **r**.
 - The **address r** is passed to **A1** which saves it in **some location** for later processing.
 - **A1** then **invokes A2** which in turn calls **A3**.
 - In each case the **calling procedure** passes the **return address** to the **called procedure**.
 - If we examine the **memory** while **A3** is computing there will be an **implicit stack** which looks like **(q,r,s,t)**.
 - The first entry, **q**, is the **address** in the **operating system** where **MAIN** returns **control**.
 - This **list operates as a stack** since the **returns will be made in the reverse order of the calls**.

- 
- Thus t is removed before s , s before r and r before q .
 - Equivalently, this means that $A3$ must finish processing before $A2$, $A2$ before $A1$, and $A1$ before MAIN.
 - This list of return addresses need not be maintained in consecutive locations.
 - For each subroutine there is usually a single location associated with the machine code which is used to retain the return address.
 - This can be severely limiting in the case of recursive calls and re-entrant routines, since every time we call a subroutine the new return address wipes out the old one.
 - For ex, if we inserted a call to $A1$ within subroutine $A3$ expecting the return to be at location u , then at execution time the stack would become (q, u, s, t) and the return address r would be lost.
 - When recursion is allowed, it is no longer adequate to reserve one location for the return address of each subroutine.
 - Since returns are made in the reverse order of calls, an elegant and natural solution to this subroutine return problem is afforded through the explicit use of a stack of return addresses.
 - Whenever a return is made, it is to the top address in the stack.

- 
- **System stack-** It is used by a **program** at run-time to **process function calls**.
 - Whenever a **function** is **invoked**, the **program** creates a **structure**, referred to as an **activation record or a stack frame**, and places it on **top** of the **system stack**.
 - Initially, the activation record for the invoked function contains only a pointer to the previous stack frame and a return address.
 - The previous stack frame pointer points to the stack frame of the invoking function, while the return address contains the location of the statement to be executed after the function terminates.
 - Since only one function executes at any given time, the function whose stack frame is on top of the system stack is chosen.
 - If this function invokes another function, the local variables, except those declared static, and the parameters of the invoking function are added to its stack frame.
 - A new stack frame is then created for the invoked function and placed on top of the system stack.
 - When this function terminates, its stack frame is removed and the processing of the invoking function, which is again on top of the stack, continues.

- 
- Assume that we have a main function that invokes function *ai*.
 - Figure 3.2(a) shows the system stack before *ai* is invoked;
 - Figure 3.2(b) shows the system stack after *ai* has been invoked.
 - Frame pointer *fp* is a pointer to the current stack frame.
 - The system also maintains separately a stack pointer, *sp*, which we have not illustrated.
 - Since all functions are stored similarly in the system stack, it makes no difference if the invoking function calls itself.
 - That is, a recursive call requires no special strategy;
 - The run-time program simply creates a new stack frame for each recursive call.
 - However, recursion can consume a significant portion of the memory allocated to the system stack; it could consume the entire available memory.

an application of stack: stack frame of function call (activation record)



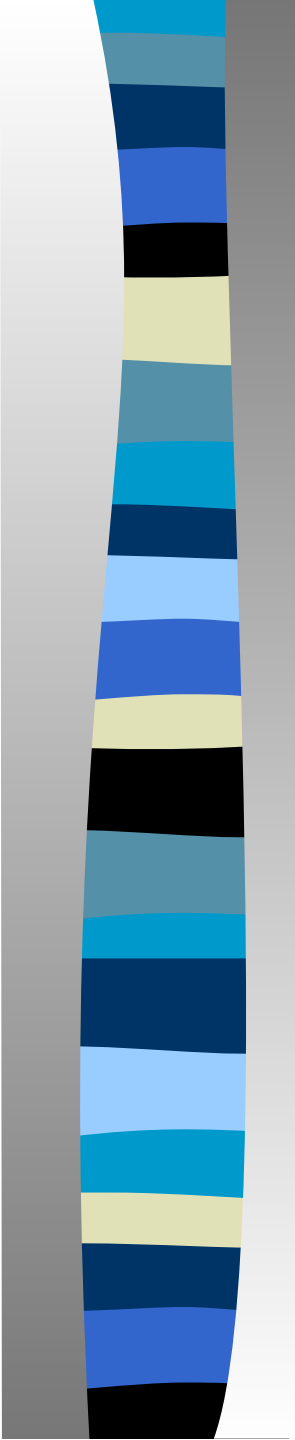
system stack before a1 is invoked

system stack after a1 is invoked

(a)

(b)

***Figure 3.2: System stack after function call `a1` (p.103)**



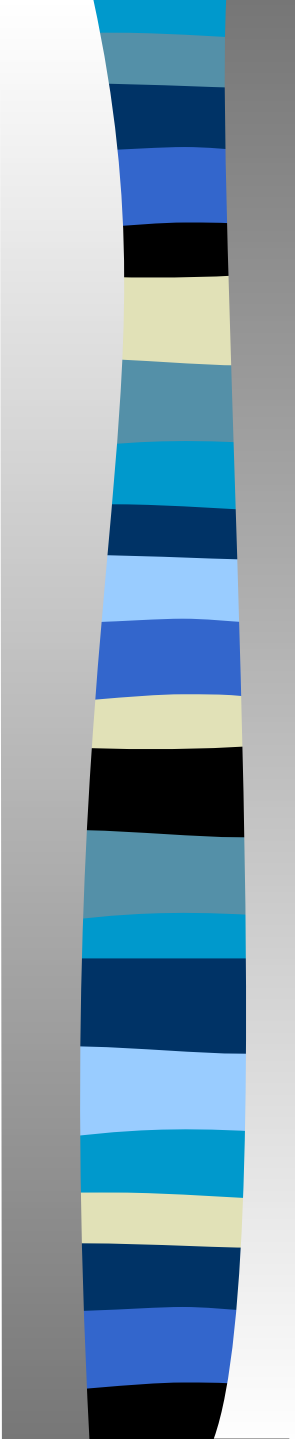
Representation-Stacks

- Associated with the object stack there are several operations that are necessary:
- CREATE (S) which creates S as an empty stack;
- ADD (i, S) which inserts the element i onto the stack S and returns the new stack;
- DELETE (S) which removes the top element of stack S and returns the new stack;
- TOP (S) which returns the top element of stack S ;
- ISEMTS (S) which returns true if S is empty else false;
- These five functions constitute a working definition of a stack.
- However we choose to represent a stack, it must be possible to build these operations.

- Formal structure of STACK.

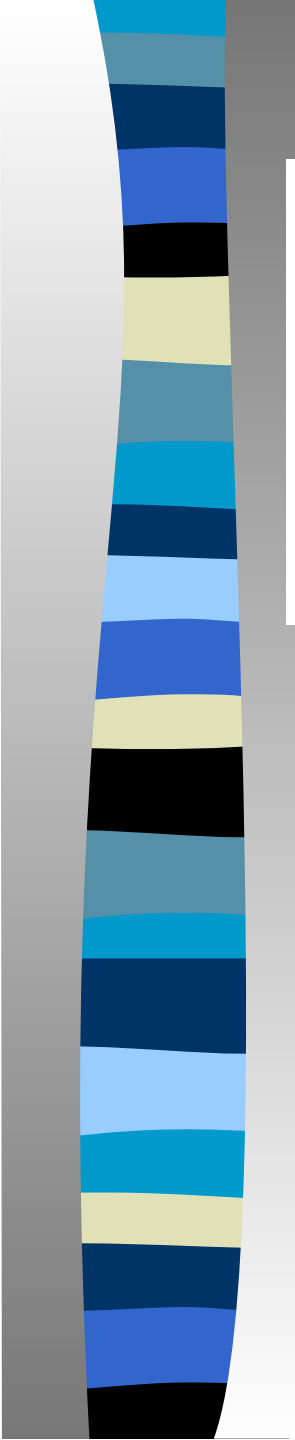
```

structure STACK (item)
1   declare CREATE( )  $\rightarrow$  stack
2       ADD(item,stack)  $\rightarrow$  stack
3       DELETE(stack)  $\rightarrow$  stack
4       TOP(stack)  $\rightarrow$  item
5       ISEMPTS(stack)  $\rightarrow$  boolean;
6   for all  $S \in \text{stack}, i \in \text{item}$  let
7       ISEMPTS (CREATE)       $:: =$  true
8       ISEMPTS (ADD(i,S))   $:: =$  false
9       DELETE (CREATE)       $:: =$  error
10      DELETE (ADD(i,S))   $:: =$  S
11      TOP (CREATE)           $:: =$  error
12      TOP (ADD(i,S))     $:: =$  i
13  end
end STACK
```

- 
- The five functions with their domains and ranges are declared in lines 1 through 5.
 - Lines 6 through 13 are the set of axioms which describe how the functions are related.
 - Lines 10 and 12 are the essential ones which define the last-in-first-out behavior.
 - The above definitions describe an infinite stack for no upper bound on the number of elements is specified.
 - This will be dealt with when we represent this structure in a computer.
 - The simplest way to represent a stack is by using a one-dimensional array, say $\text{STACK}(1:n)$, where n is the maximum number of allowable entries.
 - The first or bottom element in the stack will be stored at $\text{STACK}(1)$, the second at $\text{STACK}(2)$ and the i -th at $\text{STACK}(i)$.
 - Associated with the array will be a variable, top , which points to the top element in the stack.

Axioms

- (i) line 7: $ISEMTS(CREATE):: = \mathbf{true}$
 - Since CREATE results in top being initialized to zero, it follows from the implementation of $ISEMTS$ that $ISEMTS(CREATE):: = \mathbf{true}$.
- (ii) line 8: $ISEMTS(ADD(i,S)):: = \mathbf{false}$
 - The value of top is changed only in procedures CREATE, ADD and DELETE. CREATE initializes top to zero while ADD increments it by 1 so long as top is less than n (this is necessary because we can implement only a finite stack).
 - DELETE decreases top by 1 but never allows its value to become less than zero.
 - Hence, $ADD(i,S)$ either results in an error condition (STACK_FULL), or leaves the value of $top > 0$.
 - This then implies that $ISEMTS(ADD(i,S)):: = \mathbf{false}$.
- (iii) line 9: $DELETE(CREATE):: = error$
 - This follows from the observation that CREATE sets $top = 0$ and the procedure DELETE signals the error condition STACK_EMPTY when $top = 0$.



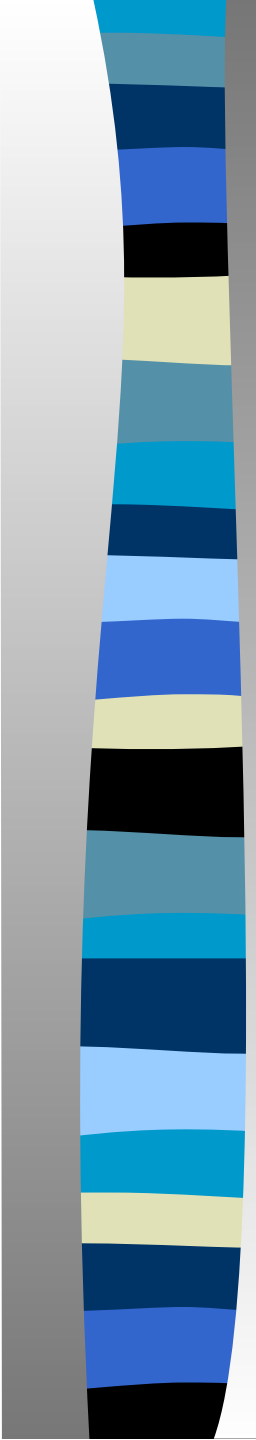
Implementation

CREATE () :: = declare STACK(1:n); top ← 0

*ISEMPTS(STACK) :: = if top = 0 then true
 else false*

*TOP(STACK) :: = if top = 0 then error
 else STACK(top)*

- The implementations of these three operations using an array are so short that we needn't make them separate procedures but can just use them directly whenever we need to.
- The ADD and DELETE operations are only a bit more complex.



procedure *ADD* (*item*, *STACK*, *n*, *top*)

 //insert item into the *STACK* of maximum size *n*; *top* is the number
 of elements currently in *STACK* //

if $top \geq n$ **then call** *STACK_FULL*

$top \leftarrow top + 1$

$STACK(top) \leftarrow item$

end *ADD*

procedure *DELETE* (*item*, *STACK*, *top*)

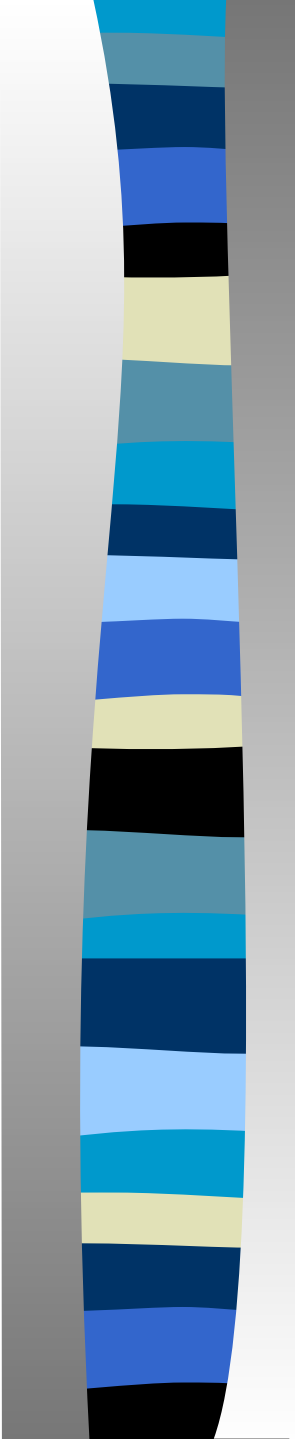
 //removes the top element of *STACK* and stores it in *item*
 unless *STACK* is empty //

if $top \leq 0$ **then call** *STACK_EMPTY*

$item \leftarrow STACK(top)$

$top \leftarrow top - 1$

end *DELETE*



Another Book Representation – Stack(ADT, OPERATIONS) BEGIN

Abstract data type for stack

structure *Stack* is

objects: a finite ordered list with zero or more elements.

functions:

for all $stack \in Stack$, $item \in element$, max_stack_size
 $\in$ positive integer

Stack CreateS(max_stack_size) ::=

create an empty stack whose maximum size is
 max_stack_size

Boolean IsFull($stack$, max_stack_size) ::=

if (number of elements in $stack == max_stack_size$)

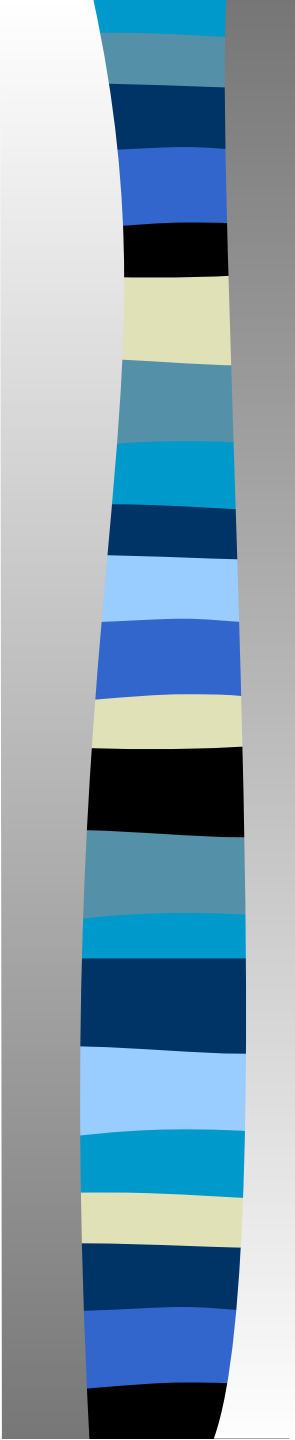
return TRUE

else return FALSE

Stack Add($stack$, $item$) ::=

if (IsFull($stack$)) $stack_full$

else insert $item$ into top of $stack$ and **return**



Boolean IsEmpty(*stack*) ::=

if(*stack* == CreateS(*max_stack_size*))

return TRUE

else return FALSE

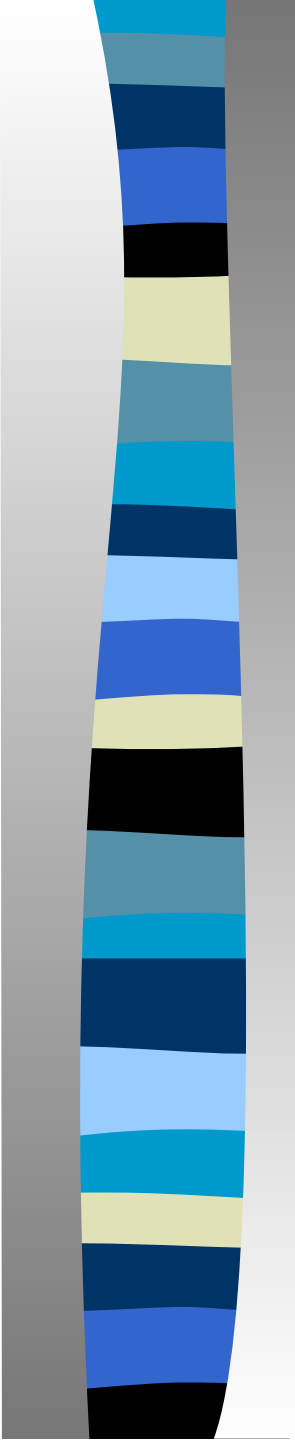
Element Delete(*stack*) ::=

if(IsEmpty(*stack*)) **return**

else remove and return the *item* on the top

 of the stack.

***Structure 3.1: Abstract data type *Stack* (p.104)**



Implementation: using array

Stack CreateS(max_stack_size) ::=

```
#define MAX_STACK_SIZE 100 /* maximum stack size */
```

```
typedef struct {
```

```
    int key;
```

```
    /* other fields */
```

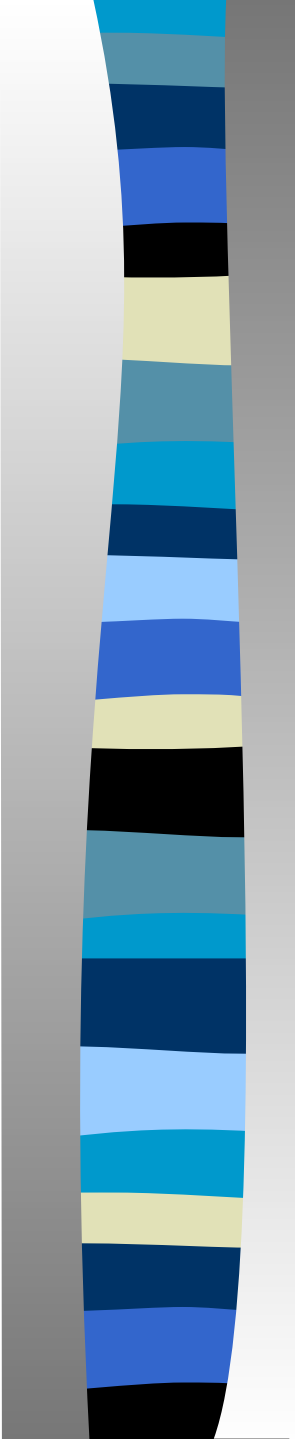
```
    } element;
```

```
element stack[MAX_STACK_SIZE];
```

```
int top = -1;
```

Boolean IsEmpty(Stack) ::= top < 0;

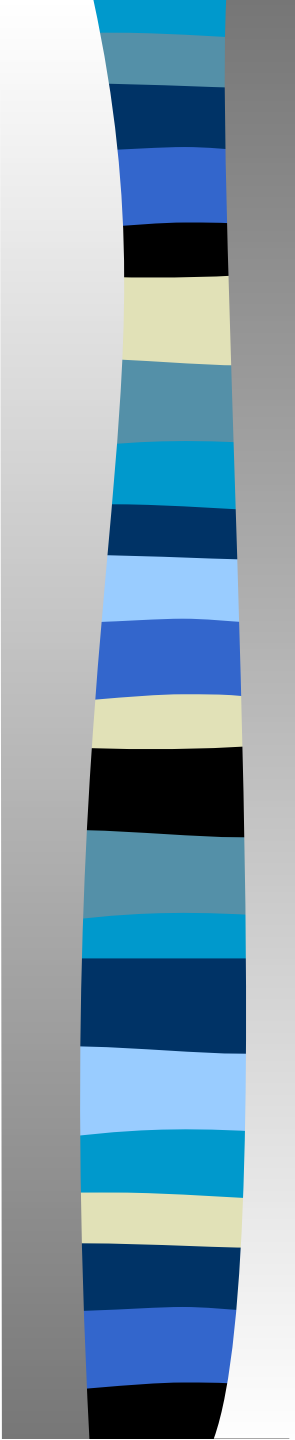
Boolean IsFull(Stack) ::= top >= MAX_STACK_SIZE-1;



Add to a stack

```
void add(int *top, element item)
{
    /* add an item to the global stack */
    if (*top >= MAX_STACK_SIZE-1) {
        stack_full( );
        return;
    }
    stack[++*top] = item;
}
```

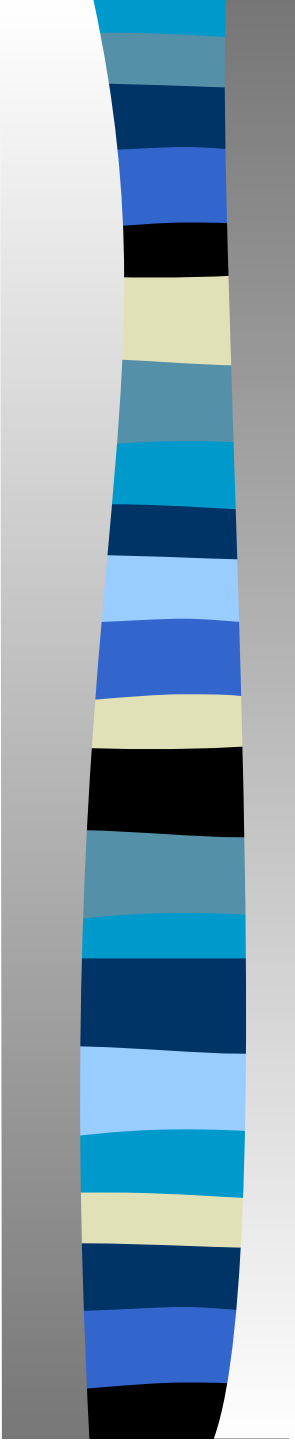
***program 3.1: Add to a stack (p.104)**



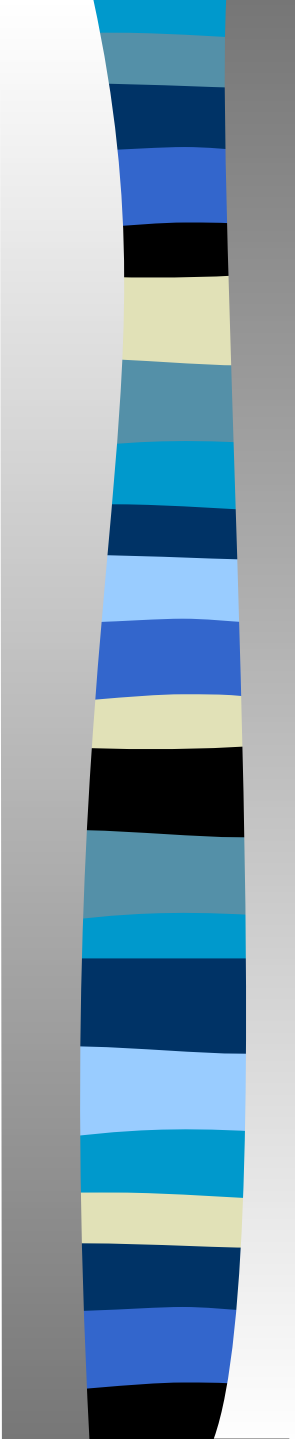
Delete from a stack

```
element delete(int *top)
{
    /* return the top element from the stack */
    if (*top == -1)
        return stack_empty( ); /* returns and error key */
    return stack[(*top)--];
}
```

***Program 3.2: Delete from a stack (p.105)**



END



Applications of Stacks

- Evaluation of an Expression
- Conversion of infix to postfix expression
- Checking whether the expression is valid or not
- Checking for a Palindrome
- Implementing rat & maze problem

Evaluation of Expressions

$$X = a / b - c + d * e - a * c$$

$$a = 4, b = c = 2, d = e = 3$$

Interpretation 1:

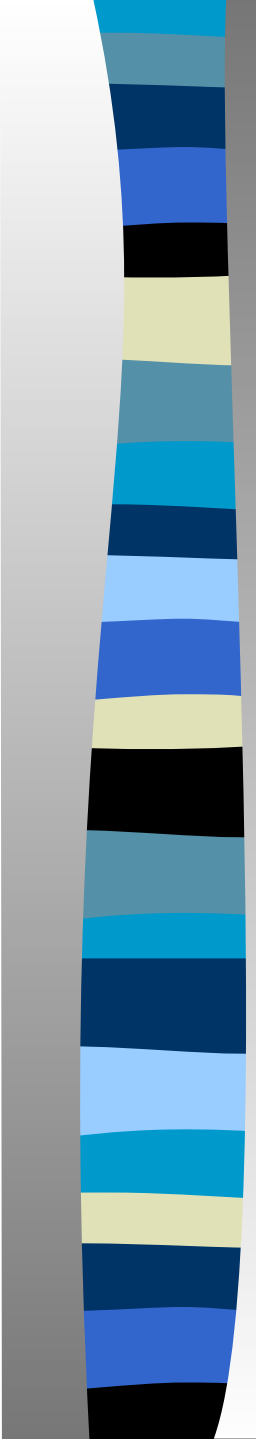
$$((4/2)-2)+(3*3)-(4*2)=0 + 8+9=1$$

Interpretation 2:

$$(4/(2-2+3))*(3-4)*2=(4/3)*(-1)*2=-2.66666\cdots$$

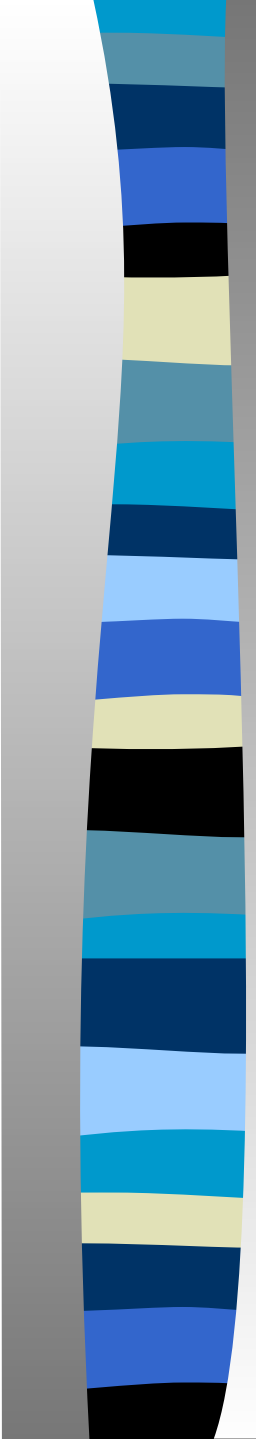
How to generate the machine instructions
corresponding to a given expression?

precedence rule + associative rule

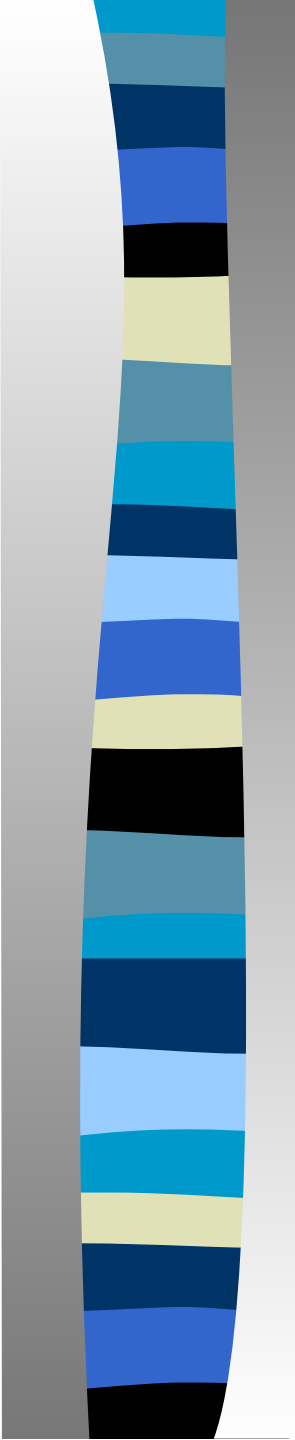


<u>Operator</u>	<u>Priority</u>
** , unary - , unary + , ¬	6
* , /	5
+ , -	4
< , <= , = , ≠ , >= , > , ≠	3
and	2
or	1

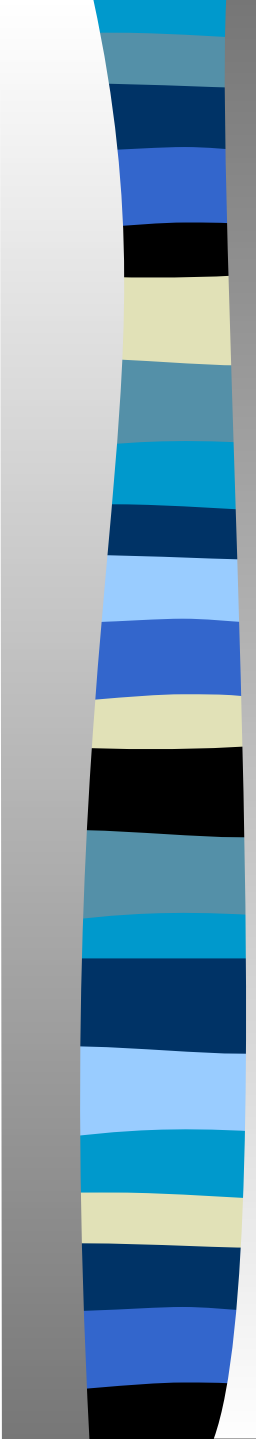
Figure 3.7. Priority of arithmetic, Boolean and relational operators



Token	Operator	Precedence ¹	Associativity
() [] -> .	function call array element struct or union member	17	left-to-right
-- ++	increment, decrement ²	16	left-to-right
-- ++ ! - - + & * sizeof	decrement, increment ³ logical not one's complement unary minus or plus address or indirection size (in bytes)	15	right-to-left
(type)	type cast	14	right-to-left
* / %	mutiplicative	13	Left-to-right



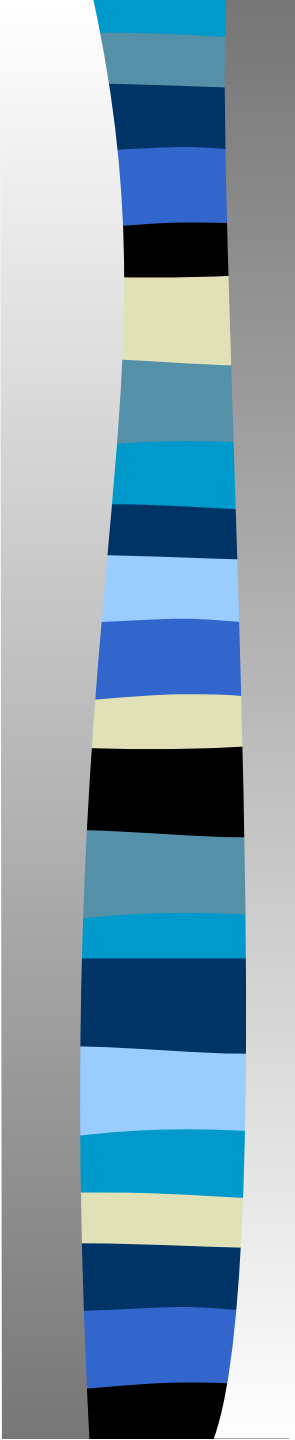
+ -	binary add or subtract	12	left-to-right
<< >>	shift	11	left-to-right
> >= < <=	relational	10	left-to-right
== !=	equality	9	left-to-right
&	bitwise and	8	left-to-right
^	bitwise exclusive or	7	left-to-right
	bitwise or	6	left-to-right
&&	logical and	5	left-to-right
⊞	logical or	4	left-to-right

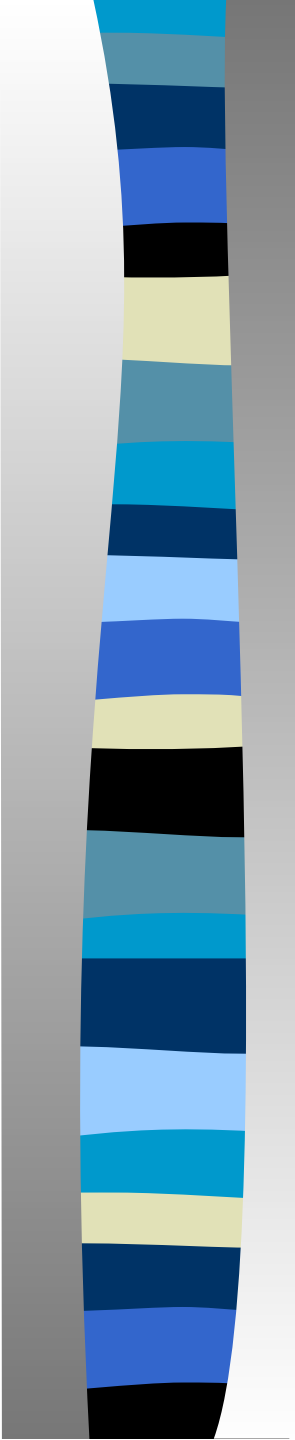


?:	conditional	3	right-to-left
= += -= /= *= %= <<= >>= &= ^=	assignment	2	right-to-left
,	comma	1	left-to-right

- 1.The precedence column is taken from Harbison and Steele.
- 2.Postfix form
- 3.prefix form

***Figure 3.12: Precedence hierarchy for C (p.119)**

- 
- The standard way of writing expressions is known as **infix notation** because in it a binary operator is placed in-between its two operands.
 - Infix notation is used for all the expressions written so far.
 - Although **infix notation** is the most common way of writing expressions, it is not the one used by compilers to evaluate expressions.
 - Instead compilers typically use a **parenthesis-free notation** referred to as **postfix**.
 - In this notation, each operator appears after its operands.
 - Figure 3.13 contains several infix expressions and their postfix equivalents.



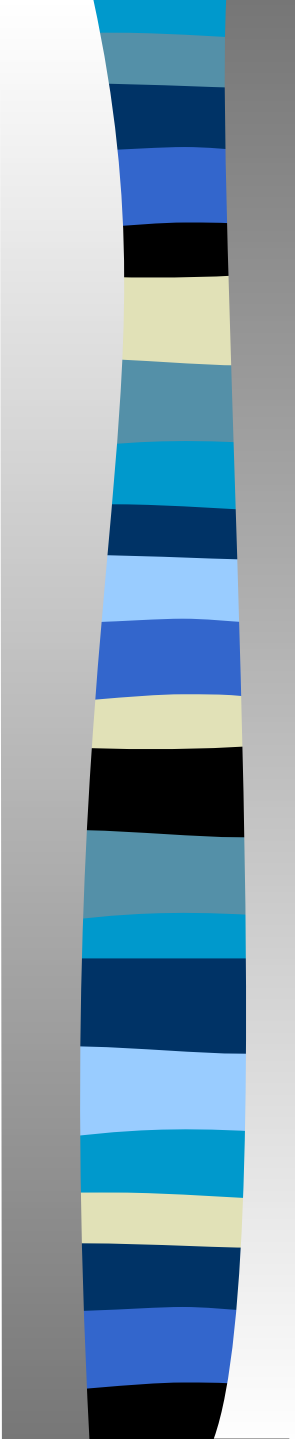
user

compiler

Infix	Postfix
2+3*4	234*+
a*b+5	ab*5+
(1+2)*7	12+7*
a*b/c	ab*c/
(a/(b-c+d))*(e-a)*c	abc-d+/ea-*c*
a/b-c+d*e-a*c	ab/c-de*ac*-

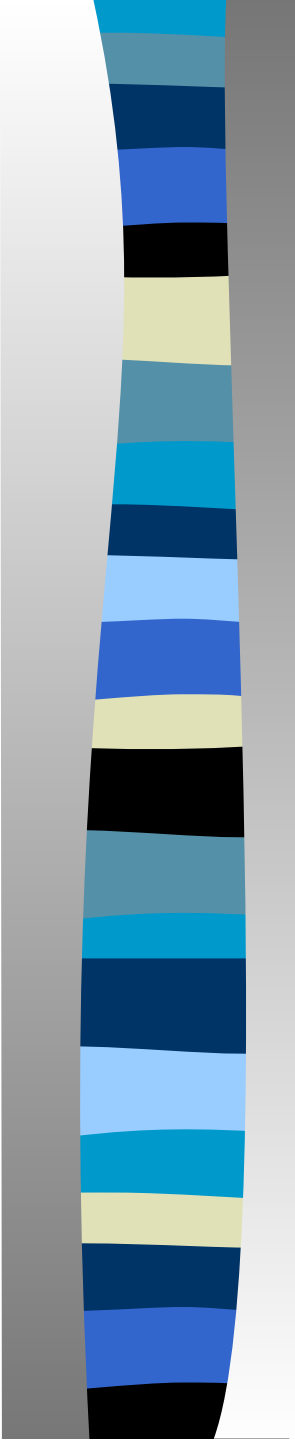
*Figure 3.13: Infix and postfix notation (p.120)

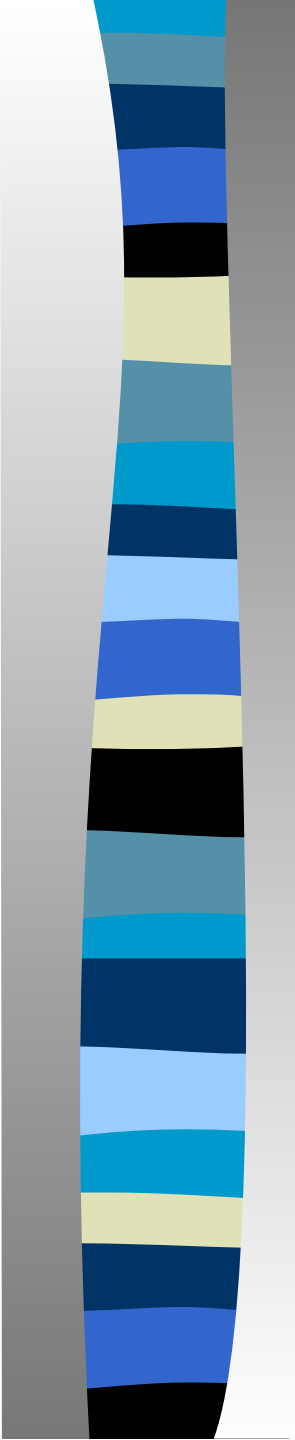
Postfix: no parentheses, no precedence

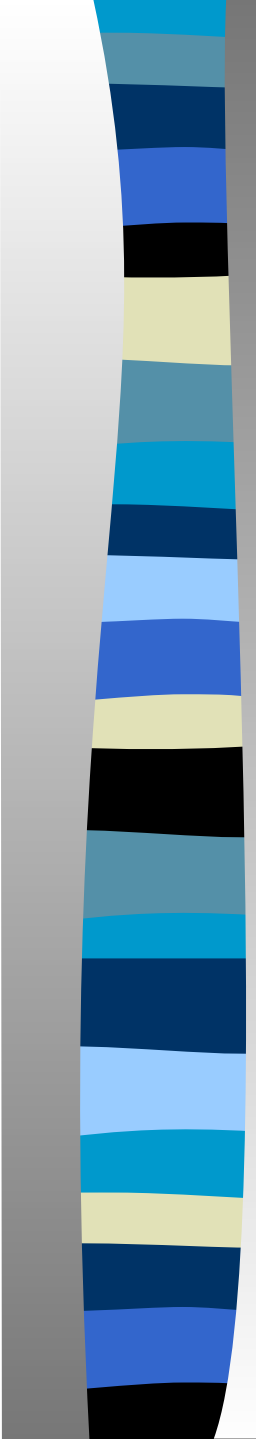
- 
- $2+3*4$
 - $2+34^*$
 - 234^*+

- $a*b+5$
 - ab^*+5
 - $ab*5+$

- $(1+2)*7$
 - $12+^*7$
 - $12+7^*$

- 
- $a*b/c$
 - ab^*/c
 - $ab^*c/$
 - $(a/(b-c+d))*(e-a)*c$
 - $(a/(bc-d+)*ea-*c$
 - $abc-d+/*ea-*c$
 - $abc-d+/ea-*c$
 - $abc-d+/ea-*c*$

- 
- $a/b - c + d * e - a * c$
 - $ab/-c + de^* - ac^*$
 - $ab/c - +de^* - ac^*$
 - $ab/c - de^* + -ac^*$
 - $ab/c - de^* + ac^* -$

- 
- Before writing a function that translates expressions from infix to postfix, we tackle the easier task of **evaluating postfix** expressions.
 - This evaluation process is much **simpler** than the evaluation of infix expressions because there are **no parentheses** to consider.
 - To evaluate an expression make a single left-to-right **scan** of it.
 - **Place/push the operands** on a stack until an operator is found.
 - If an **operator** is found **remove/pop** the correct number of operands from the stack, for that operator.
 - Perform the operation, and **place/push** the result back on the stack.
 - Continue the same fashion until the end of the expression is reached.
 - Remove the answer from the top of the stack.

Postfix $\rightarrow$ a b c * + d - Let, a = 4, b = 3, c = 2, d = 5

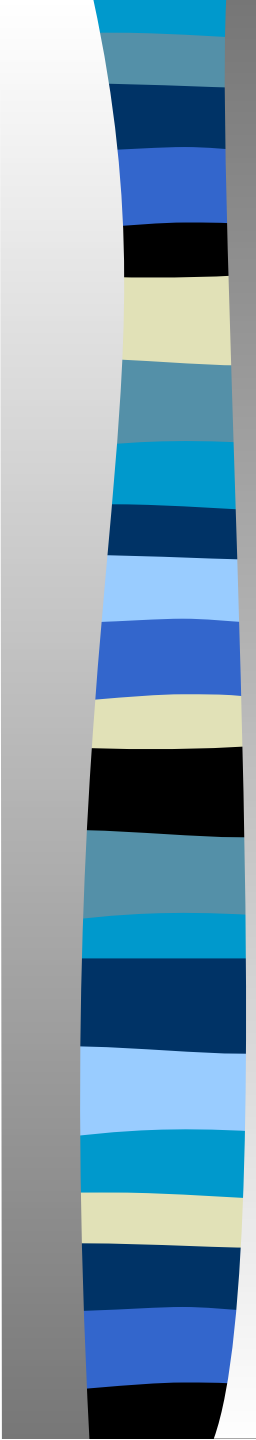
Postfix $\rightarrow$ 4 3 2 * + 5 -

Operator/Operand	Action	Stack
4	Push	4
3	Push	4, 3
2	Push	4, 3, 2
*	Pop (2, 3) and $3 * 2 = 6$ then Push 6	4, 6
+	Pop (6, 4) and $4 + 6 = 10$ then Push 10	10
5	Push	10, 5
-	Pop (5, 10) and $10 - 5 = 5$ then Push 5	5

- Figure 3.14 shows this processing when the input is the nine character string 6 2/3-4 2*+.

Token	Stack			Top
	[0]	[1]	[2]	
6	6			0
2	6	2		1
/	6/2			0
3	6/2	3		1
-	6/2-3			0
4	6/2-3	4		1
2	6/2-3	4	2	2
*	6/2-3	4*2		1
+	6/2-3+4*2			0

***Figure 3.14:** Postfix evaluation (p.120)



procedure *EVAL* (*E*)

 // evaluate the postfix expression *E*. It is assumed that the last character in *E* is an ' ∞ '. A procedure *NEXT-TOKEN* is used to extract from *E* the next token. A token is either an operand, operator, or ' ∞ '. A one dimensional array *STACK*(1:*n*) is used as a stack //

$top \leftarrow 0$ // initialize *STACK* //

loop

$x \leftarrow \text{NEXT-TOKEN}(E)$

case

 : $x = '\infty'$: **return** // answer is at top of stack //

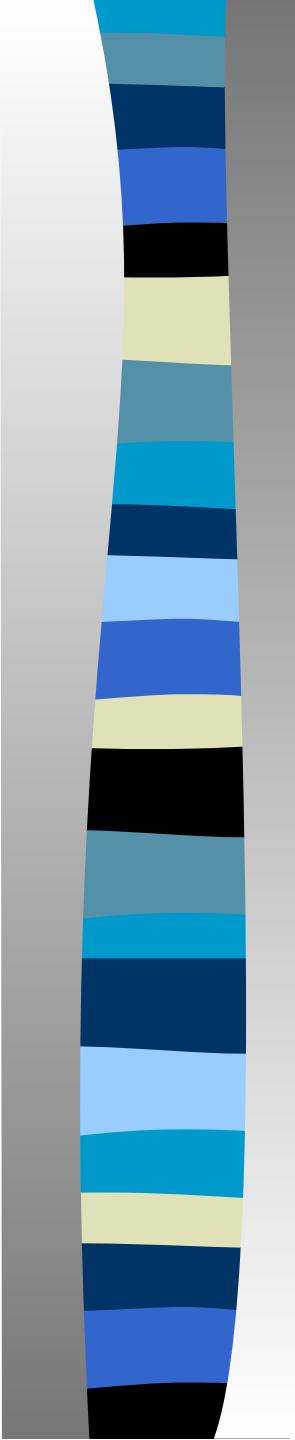
 : x is an operand: **call** *ADD*(x, STACK, n, top)

 : **else**: *remove the correct number of operands for operator x from STACK, perform the operation and store the result, if any, onto the stack*

end

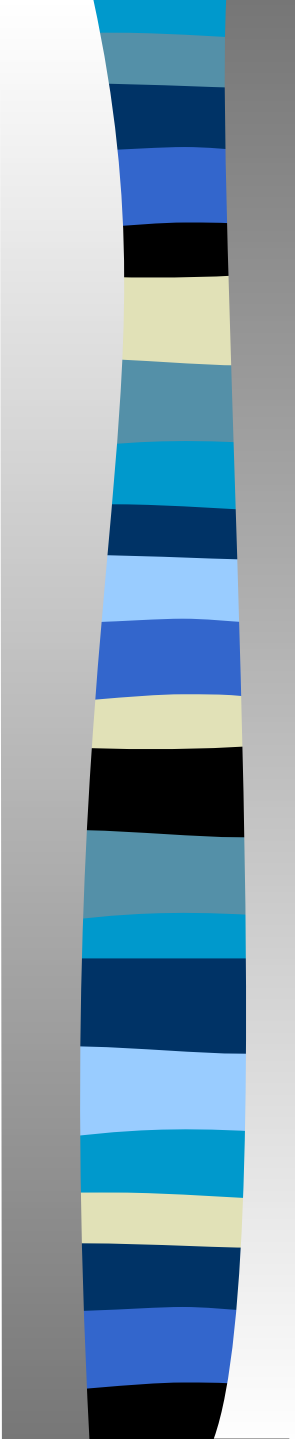
forever

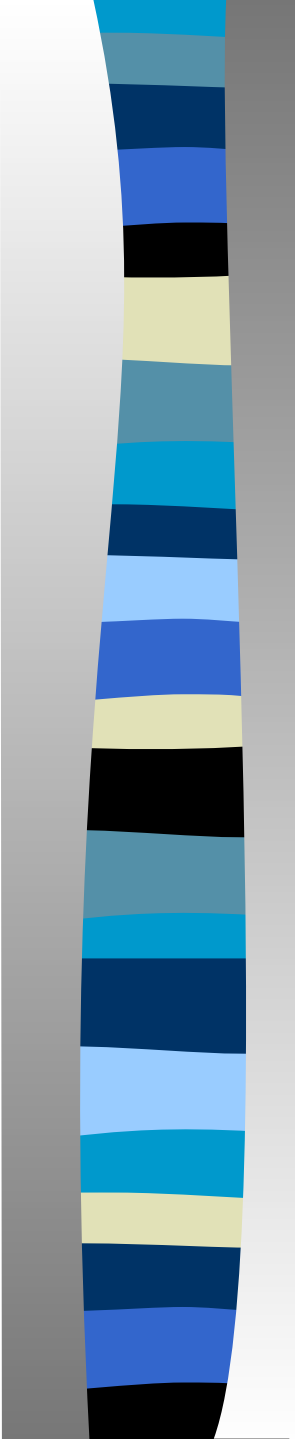
end *EVAL*



Summary - Evaluation of an postfix expression

- Begin
- For each character in postfix exp do
 - If operand is encountered, push it onto the stack
 - Else if operator is encountered, pop 2 elements
 - A-->top element
 - B-->next to top element
 - Result=B operator A
 - Push result onto stack
 - Return element of stack top
- end

- 
- Another Book Representation- Begin



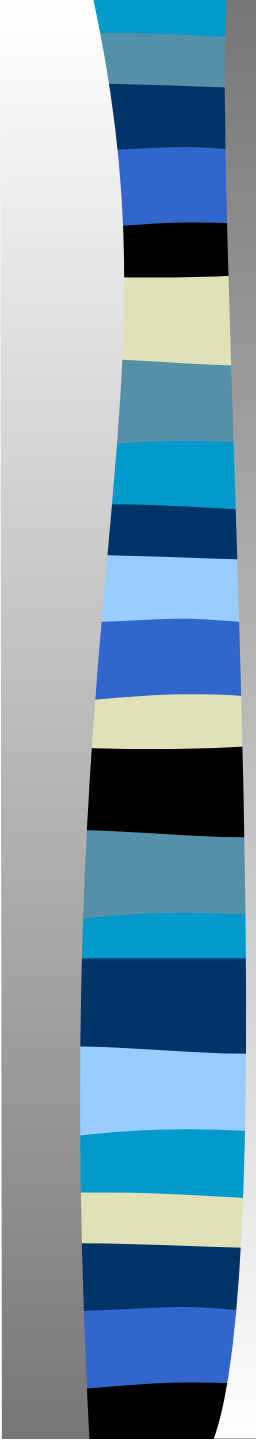
Goal: infix --> postfix

Assumptions:

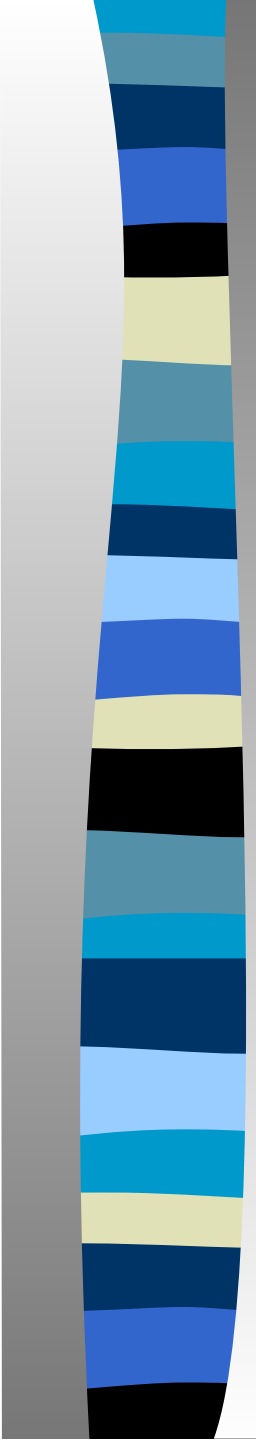
operators: +, -, *, /, %

operands: single digit integer

```
#define MAX_STACK_SIZE 100 /* maximum stack size */
#define MAX_EXPR_SIZE 100 /* max size of expression */
typedef enum {lparan, rparen, plus, minus, times, divide,
              mod, eos, operand} precedence;
int stack[MAX_STACK_SIZE]; /* global stack */
char expr[MAX_EXPR_SIZE]; /* input string */
```

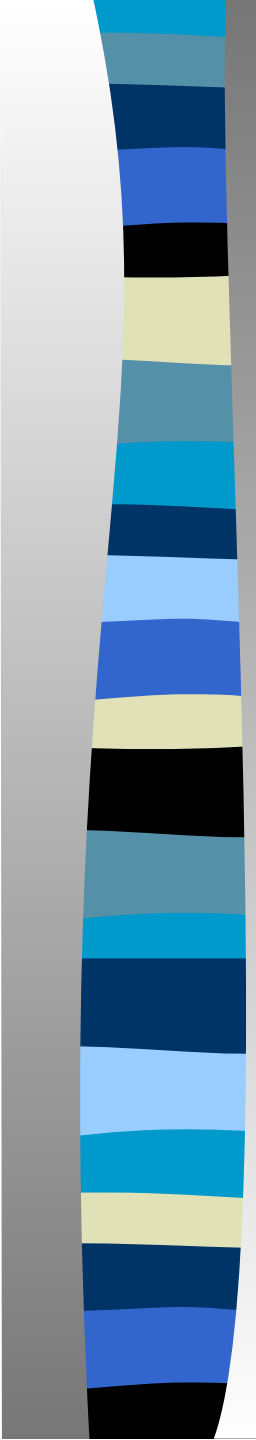


```
int eval(void)
{
/* evaluate a postfix expression, expr, maintained as a
   global variable, '\0' is the the end of the expression.
   The stack and top of the stack are global variables.
   get_token is used to return the token type and
   the character symbol. Operands are assumed to be single
   character digits */
precedence token;
char symbol;
int op1, op2;
int n = 0; /* counter for the expression string */
int top = -1;
token = get_token(&symbol, &n);
while (token != eos) {
    if (token == operand)           exp: character array
        add(&top, symbol-'0'); /* stack insert */
}
```



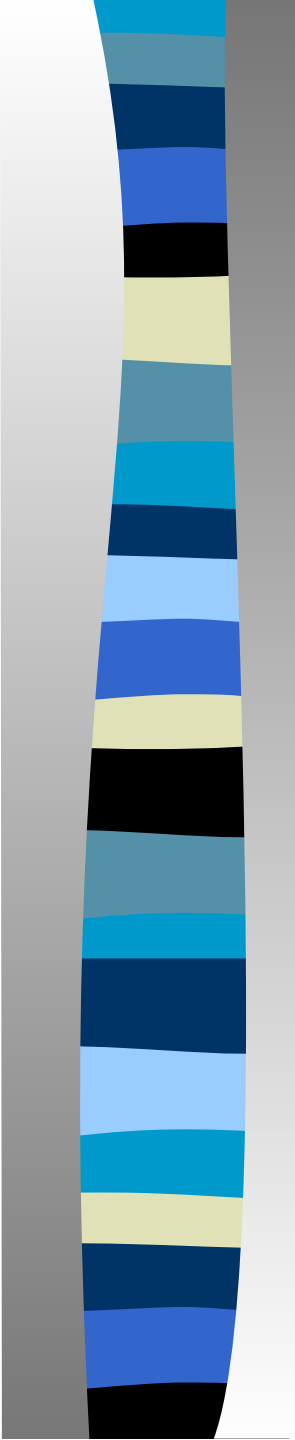
```
else {
    /* remove two operands, perform operation, and
       return result to the stack */
    op2 = delete(&top); /* stack delete */
    op1 = delete(&top);
    switch(token) {
        case plus: add(&top, op1+op2); break;
        case minus: add(&top, op1-op2); break;
        case times: add(&top, op1*op2); break;
        case divide: add(&top, op1/op2); break;
        case mod: add(&top, op1%op2);
    }
}
token = get_token (&symbol, &n);
}
return delete(&top); /* return result */
}
```

***Program 3.9:** Function to evaluate a postfix expression (p.122)



```
precedence get_token(char *symbol, int *n)
{
    /* get the next token, symbol is the character
       representation, which is returned, the token is
       represented by its enumerated value, which
       is returned in the function name */
```

```
    *symbol =expr[(*n)++];
    switch (*symbol) {
        case '(' : return lparen;
        case ')' : return rparen;
        case '+' : return plus;
        case '-' : return minus;
```



```
case '/' : return divide;
case '*' : return times;
case '%' : return mod;
case '\0' : return eos;
default : return operand;
        /* no error checking, default is operand */
    }
}
```

***Program 3.10:** Function to get a token from the input string (p.123)

Infix to Postfix Conversion (Intuitive Algorithm)

(1) Fully parenthesize expression

$$a / b - c + d * e - a * c \rightarrow \\ (((a / b) - c) + (d * e)) - a * c))$$

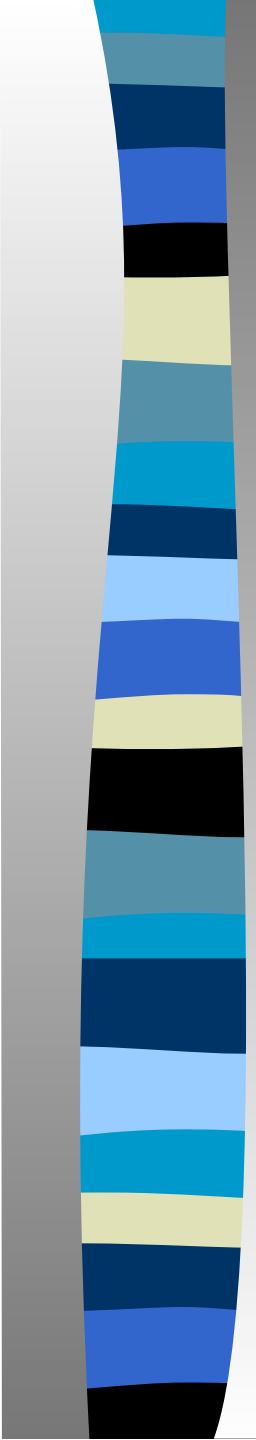
(2) All operators replace their corresponding right parentheses.

$$(((a / b) - c) + (d * e)) - a * c))$$


(3) Delete all parentheses.

$$ab/c-de^*+ac^*-$$

two passes

- 
- The problem with this as an algorithm is that it requires **two passes**:
 - the first one reads the expression and parenthesizes it while the second actually moves the operators.
 - As we have already observed, the order of the operands is the **same** in infix and postfix.
 - So as we scan an expression for the first time, we can form the postfix by **immediately passing any operands to the output**.
 - Then it is just a matter of handling the **operators**.
 - The solution is to store them in a **stack** until just the right moment and then to **unstack** and **pass** them to the **output**.
 - For example, since we want $A + B * C$ to yield $ABC * +$ our algorithm should perform the following sequence of stacking (these stacks will grow to the right):

The orders of operands in infix and postfix are the same.

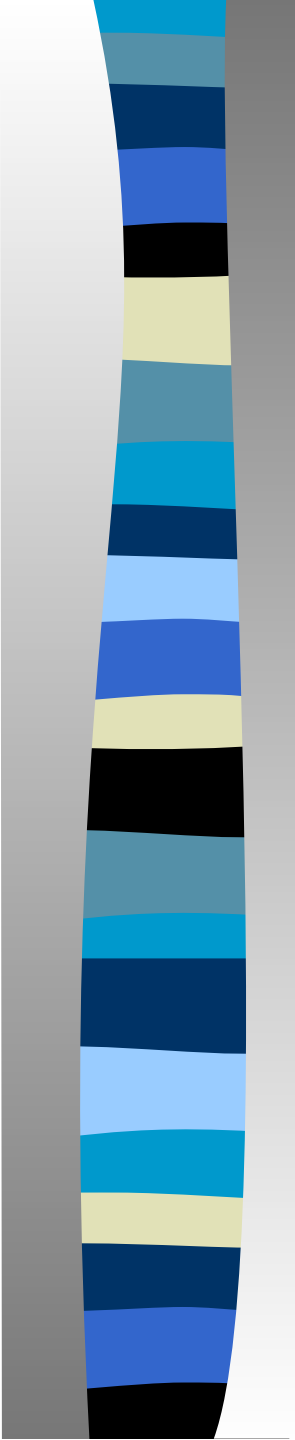
$a + b * c$, $* > +$

Token	Stack			Top	Output
	[0]	[1]	[2]		
a				-1	a
+	+			0	a
b	+			0	ab
*	+	*		1	ab
c	+	*		1	abc
eos				-1	abc*+

***Figure 3.15:** Translation of $a+b*c$ to postfix (p.124)

$$a * _1 (b + c) * _2 d$$

Token	Stack			Top	Output
	[0]	[1]	[2]		
a				-1	a
* ₁	* ₁			0	a
(	* ₁	(		1	a
b	* ₁	(		1	ab
+	* ₁	(	+	2	ab
c	* ₁	(	+	2	abc
)	* ₁	match)		0	abc+
* ₂	* ₂	* ₁ = * ₂		0	abc+* ₁
d	* ₂			0	abc+* ₁ d
eos	* ₂			0	abc+* ₁ d* ₂



Rules

- (1) Operators are taken out of the stack as long as their in-stack precedence is higher than or equal to the incoming precedence of the new operator.
- (2) (has low in-stack precedence, and high incoming precedence

Incoming pre $>$ in-stack Pre = Push

Incoming pre $<$ in-stack Pre = Pop

Incoming pre $<$ new in-stack Pre = Pop

procedure *POSTFIX* (*E*)

//convert the infix expression *E* to postfix. Assume the last character of *E* is a ' ∞ ', which will also be the last character of the postfix. Procedure *NEXT-TOKEN* returns either the next operator, operand or delimiter—whichever comes next. *STACK*(1:*n*) is used as a stack and the character ' $-\infty$ ' with $ISP('-\infty') = -1$ is used at the bottom of the stack. *ISP* and *ICP* are functions. //

STACK(1) $\leftarrow$ ' $-\infty$ '; *top* $\leftarrow$ 1 // initialize stack //

loop

x $\leftarrow$ *NEXT-TOKEN* (*E*)

case

:*x* = ' ∞ ': **while** *top* > 1 **do** // empty the stack //
 print (*STACK*(*top*)); *top* $\leftarrow$ *top* - 1
 end
 print (' ∞ ')
 return

:*x* is an operand: **print** (*x*)

:*x* = '(': **while** *STACK*(*top*) $\neq$ '(' **do** // unstack until '(' //
 print (*STACK*(*top*)); *top* $\leftarrow$ *top* - 1
 end
 top $\leftarrow$ *top* - 1 // delete '(' //

:**else**: **while** *ISP*(*STACK*(*top*)) $\geq$ *ICP*(*x*) **do**
 print (*STACK*(*top*)); *top* $\leftarrow$ *top* - 1
 end

call *ADD*(*x*, *STACK*, *n*, *top*) // insert *x* in *STACK* //

end

forever

end *POSTFIX*

loop

$x \leftarrow \text{NEXT-TOKEN}(E)$

case

: $x = '\infty'$: **while** $top > 1$ **do** // empty the stack //
 print ($STACK(top)$); $top \leftarrow top - 1$

end

print (' ∞ ')
 return

: x is an operand: **print** (x)

: $x = ')'$: **while** $STACK(top) \neq '('$ **do** // unstack until ' $'$ //
 print ($STACK(top)$); $top \leftarrow top - 1$

end

$top \leftarrow top - 1$ // delete ' $'$ //

:**else:** **while** $ISP(STACK(top)) \geq ICP(x)$ **do**

print ($STACK(top)$); $top \leftarrow top - 1$

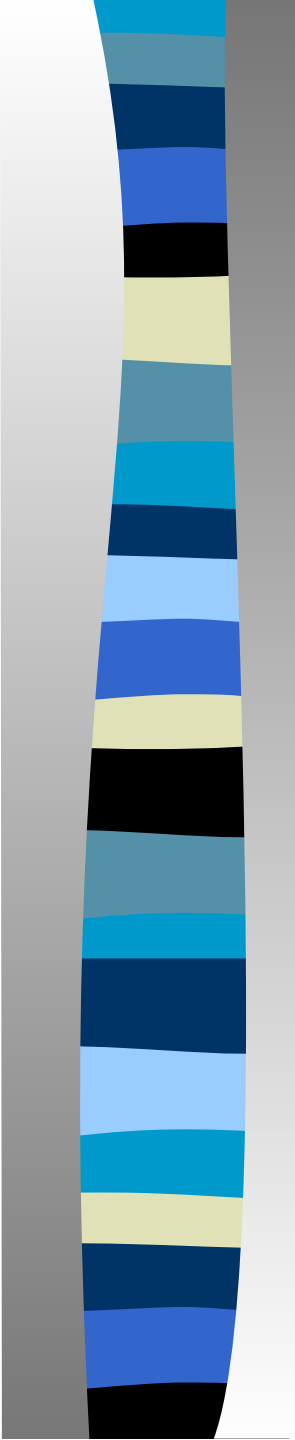
end

call $ADD(x, STACK, n, top)$ // insert x in $STACK$ //

end

forever

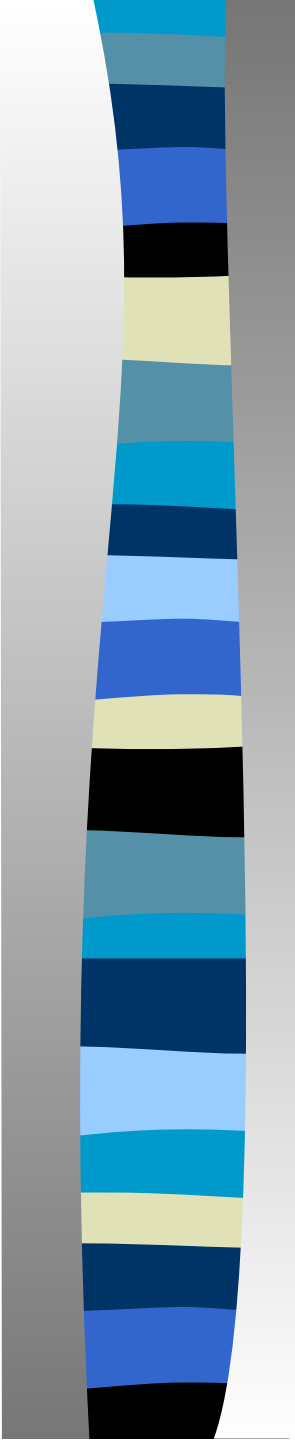
end POSTFIX



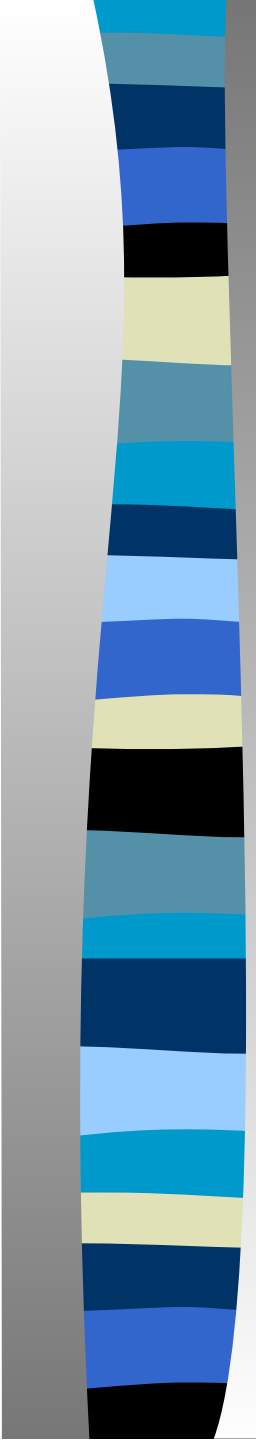
```
precedence stack[MAX_STACK_SIZE];  
/* isp and icp arrays -- index is value of precedence  
lparen, rparen, plus, minus, times, divide, mod, eos */  
static int isp [ ] = {0, 19, 12, 12, 13, 13, 13, 0};  
static int icp [ ] = {20, 19, 12, 12, 13, 13, 13, 0};
```

isp: in-stack precedence

icp: incoming precedence



```
void postfix(void)
{
    /* output the postfix of the expression. The expression
       string, the stack, and top are global */
    char symbol;
    precedence token;
    int n = 0;
    int top = 0; /* place eos on stack */
    stack[0] = eos;
    for (token = get_token(&symbol, &n); token != eos;
         token = get_token(&symbol, &n)) {
        if (token == operand)
            printf ("%c", symbol);
        else if (token == rparen ) {
```

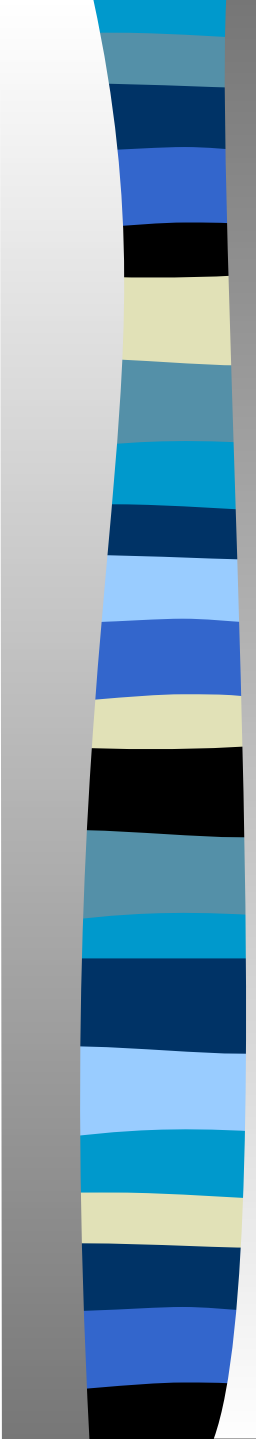


```
/*unstack tokens until left parenthesis */
while (stack[top] != lparen)
    print_token(delete(&top));
delete(&top); /*discard the left parenthesis */
}
else{
    /* remove and print symbols whose isp is greater
       than or equal to the current token's icp */
    while(isp[stack[top]] >= icp[token] )
        print_token(delete(&top));
    add(&top, token);
}
}
while ((token = delete(&top)) != eos)
    print_token(token);
print("\n");
}
```

$\theta(n)$

CHAPTER 3

***Program 3.11: Function to convert from infix to postfix (p.126)**

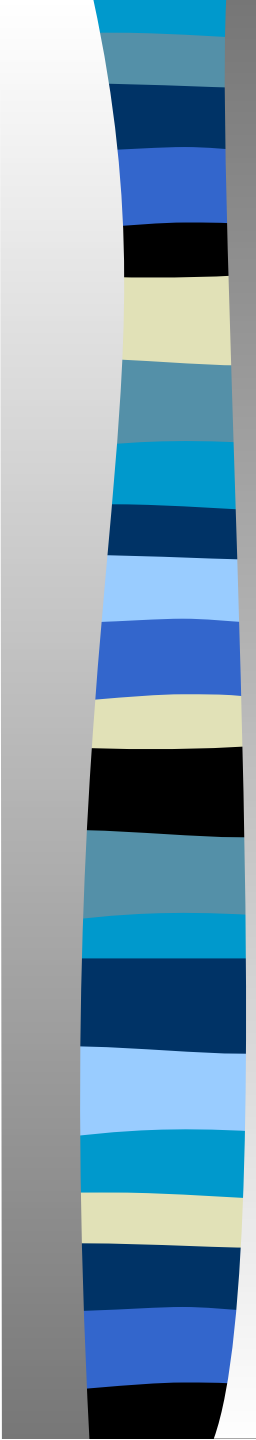


Infix	Prefix
$a*b/c$	<u>$/*abc$</u>
$a/b-c+d*e-a*c$	<u>$-+-/abc*de*ac$</u>
$a*(b+c)/d-g$	<u><u>$-/*a+bc$</u>dg</u>

(1) evaluation

(2) transformation

***Figure 3.17: Infix and postfix expressions (p.127)**



Video Links

- Stack-operations - <https://youtu.be/bxRVz8zk1WM>
- Evaluation of Expression
- <https://youtu.be/3wGTlsLnZE4>
- <https://unacademy.com/lesson/evaluation-of-a-postfix-expression-in-tabular-form/8Z5PDFL5>
<https://www.youtube.com/watch?v=3wGTlsLnZE4>
- <https://images.app.goo.gl/tXgsTaC3vyCcigpQA>
- <https://images.app.goo.gl/1fvuj7yraQ8oLUfu5>